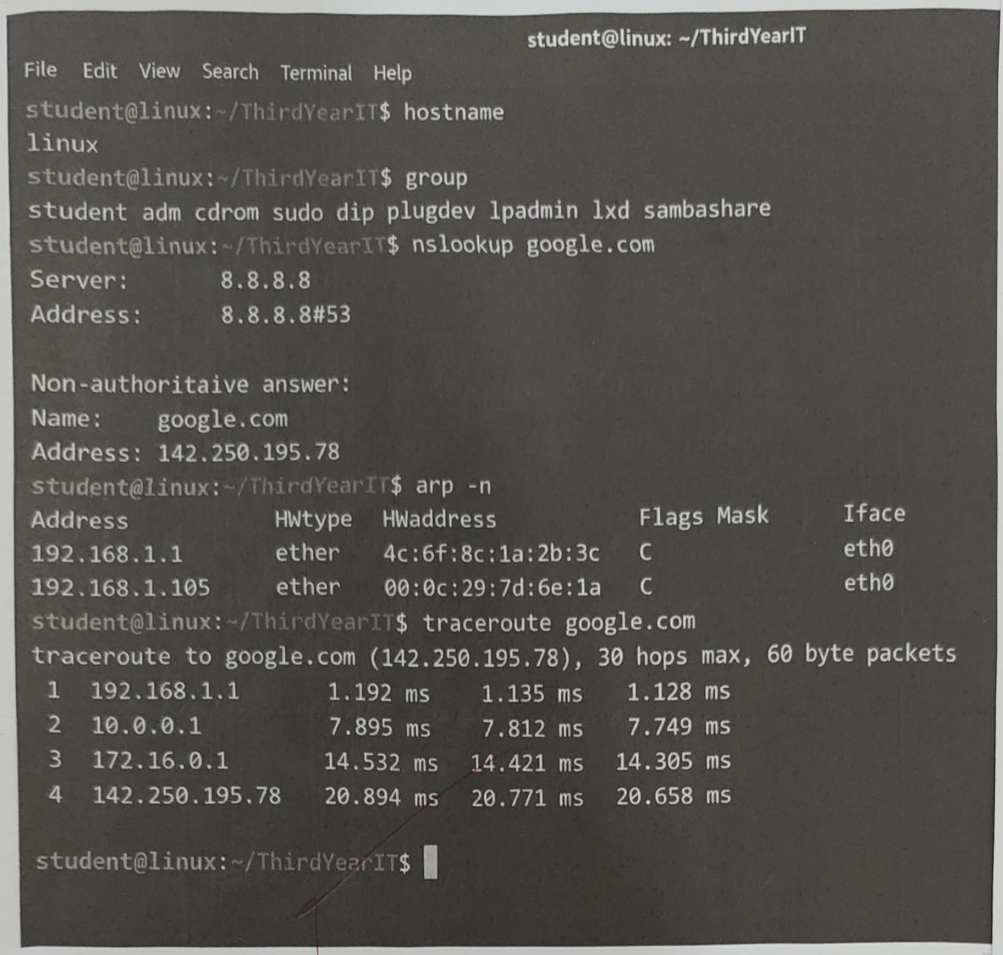
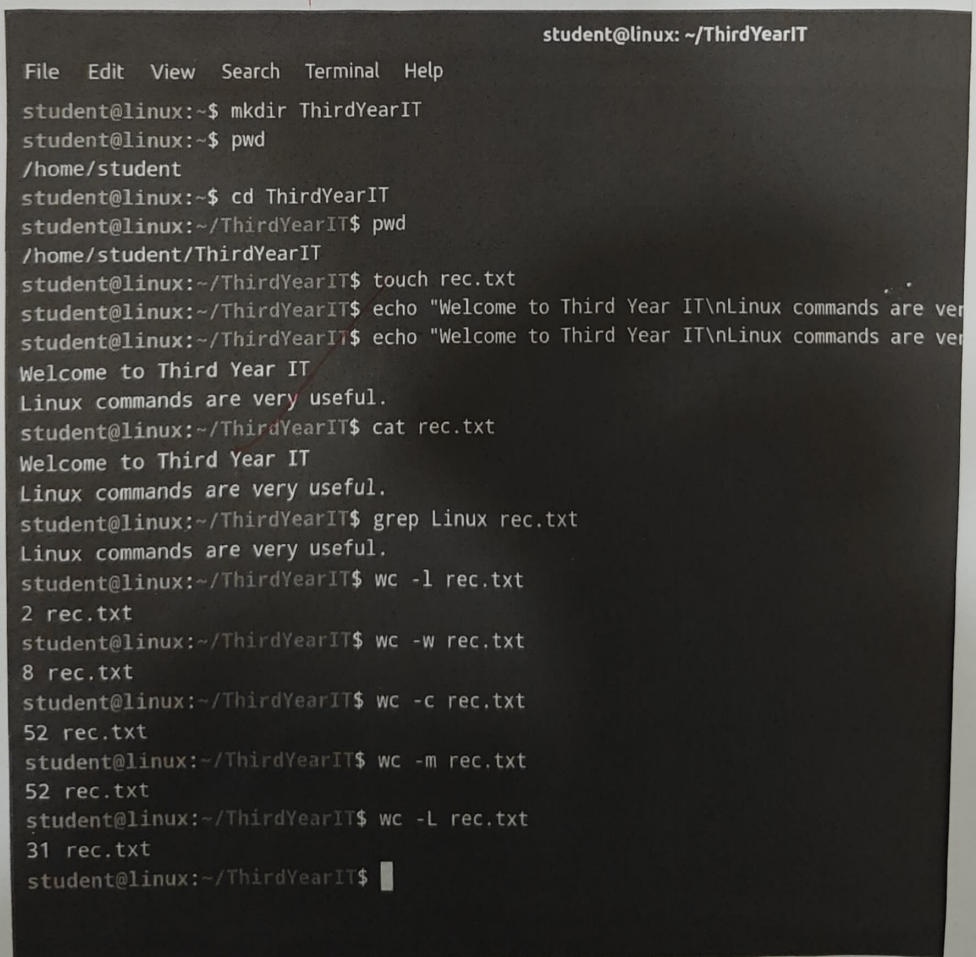

student@linux: ~/ThirdYearIT
File Edit View Search Terminal Help
student@linux:~/ThirdYearIT$ hostname
linux
student@linux:~/ThirdYearIT$ group
student adm cdrom sudo dip plugdev lpadmin lxd sambashare
student@linux:~/ThirdYearIT$ nslookup google.com
Server: 8.8.8.8
Address: 8.8.8.8#53
Non-authoritaive answer:
Name: google.com
Address: 142.250.195.78
student@linux:~/ThirdYearIT$ arp -n
Address HWtype HWaddress Flags Mask Iface
192.168.1.1 ether 4c:6f:8c:1a:2b:3c C eth0
192.168.1.105 ether 00:0c:29:7d:6e:1a C eth0
student@linux:~/ThirdYearIT$ traceroute google.com
traceroute to google.com (142.250.195.78), 30 hops max, 60 byte packets
1 192.168.1.1 1.192 ms 1.135 ms 1.128 ms
2 10.0.0.1 7.895 ms 7.812 ms 7.749 ms
3 172.16.0.1 14.532 ms 14.421 ms 14.305 ms
4 142.250.195.78 20.894 ms 20.771 ms 20.658 ms
student@linux:~/ThirdYearIT$


student@linux: ~/ThirdYearIT
File Edit View Search Terminal Help
student@linux:~$ mkdir ThirdYearIT
student@linux:~$ pwd
/home/student
student@linux:~$ cd ThirdYearIT
student@linux:~/ThirdYearIT$ pwd
/home/student/ThirdYearIT
student@linux:~/ThirdYearIT$ touch rec.txt
student@linux:~/ThirdYearIT$ echo "Welcome to Third Year IT\nLinux commands are ver
student@linux:~/ThirdYearIT$ echo "Welcome to Third Year IT\nLinux commands are ver
Welcome to Third Year IT
Linux commands are very useful.
student@linux:~/ThirdYearIT$ cat rec.txt
Welcome to Third Year IT
Linux commands are very useful.
student@linux:~/ThirdYearIT$ grep Linux rec.txt
Linux commands are very useful.
student@linux:~/ThirdYearIT$ wc -l rec.txt
2 rec.txt
student@linux:~/ThirdYearIT$ wc -w rec.txt
8 rec.txt
student@linux:~/ThirdYearIT$ wc -c rec.txt
52 rec.txt
student@linux:~/ThirdYearIT$ wc -m rec.txt
52 rec.txt
student@linux:~/ThirdYearIT$ wc -L rec.txt
31 rec.txt
student@linux:~/ThirdYearIT$

Exp :- 2
Date :- 16/12/26

Study of different types of Network cables and crimping of cables with RJ45 connector.

Aim :-

To study the different types of network cable

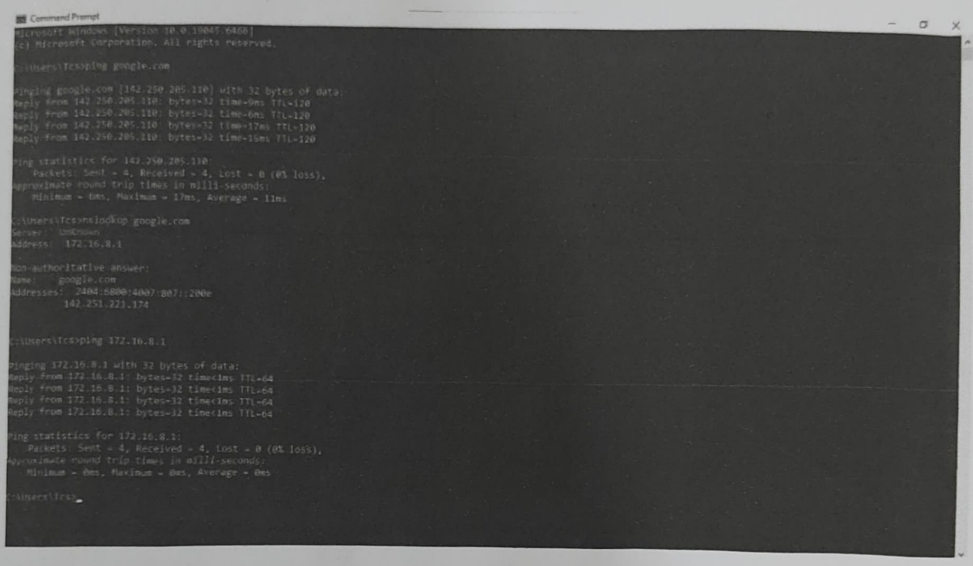
a) understanding different types of network cables
b) Shielded twisted pairs cable
3) Coaxial cable
4) fibre optic cable

Cable Type	Category	Max data transmissi.	Adv/Dis	Applicati.
UTP	Category 3, 5, 5e	10bps upto to 100 Mbps	-> Cheaper -> Easy to install	10 base ethernet
STP	Category 6	10Gbps	-> Shielded -> faster than UTP	Gigabit ethernet
Coaxial Cable	RG-6 RG-59 RG-11	10-100 Mbps	-> High bandwidth -> resistible -> limited distance	Speed of signal is 50 an

Ex:- 03 Develop a customised ping Command

18/7/26 to test the server connectivity

Aim:-



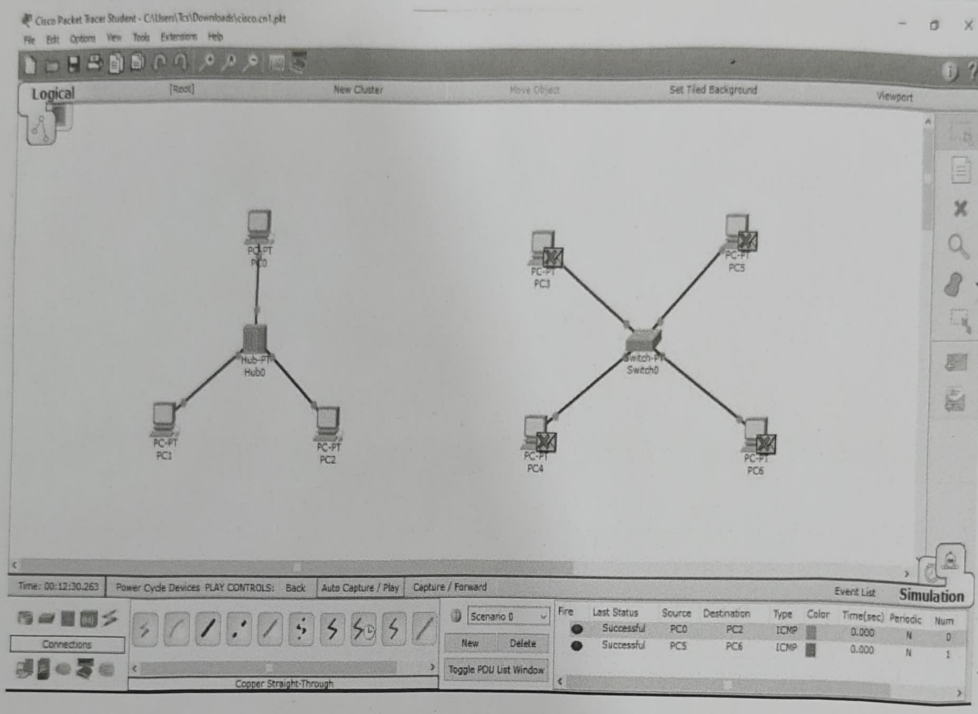
Result:-

Thus, the ping Command to test the server connectivity is developed successfully.

Exp no:- 4 Packet tracer tool Installation

22/7/26 and user Interface overview

Aim :-



Result :-

Thus, the packet tracer tool Installation of user interface overview is executed successfully.

EXPERIMENT 5: IMPLEMENT ECHO USING PYTHON

30/7/2026

Aim:

To implement echo using python

```
import socket

HOST = '127.0.0.1'   # Localhost
PORT = 12345

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(1)

print("Server is waiting for connection...")

conn, addr = server.accept()
print("Connected by:", addr)

while True:
    data = conn.recv(1024).decode()

    if not data or data.lower() == "exit":
        print("Connection closed.")
        break

    print("Client:", data)
    conn.send(data.encode())   # Echo back the same message

conn.close()
server.close()
```

```
import socket

HOST = '127.0.0.1'
PORT = 12345

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))

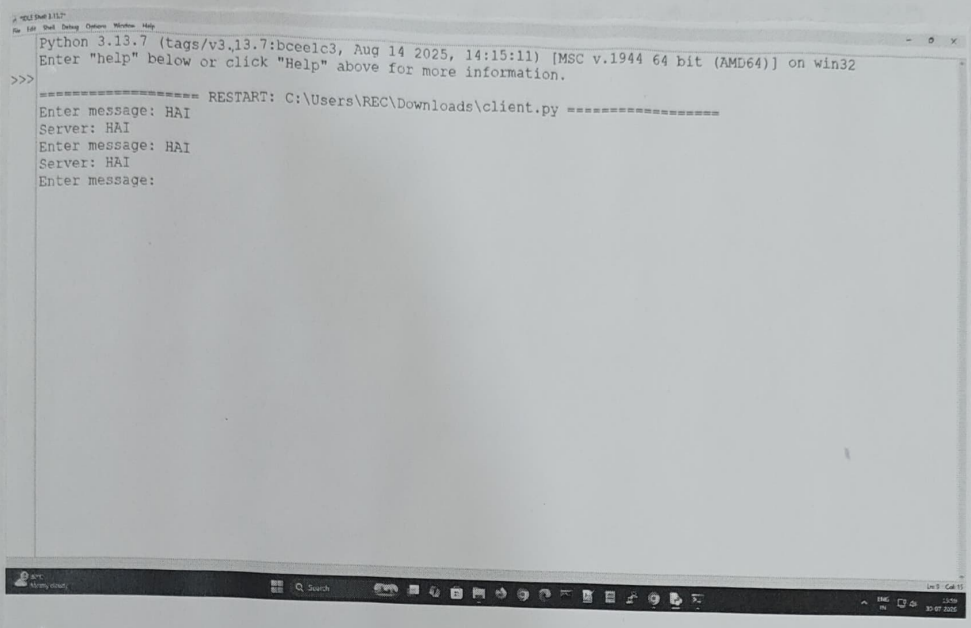
while True:
    message = input("Enter message: ")

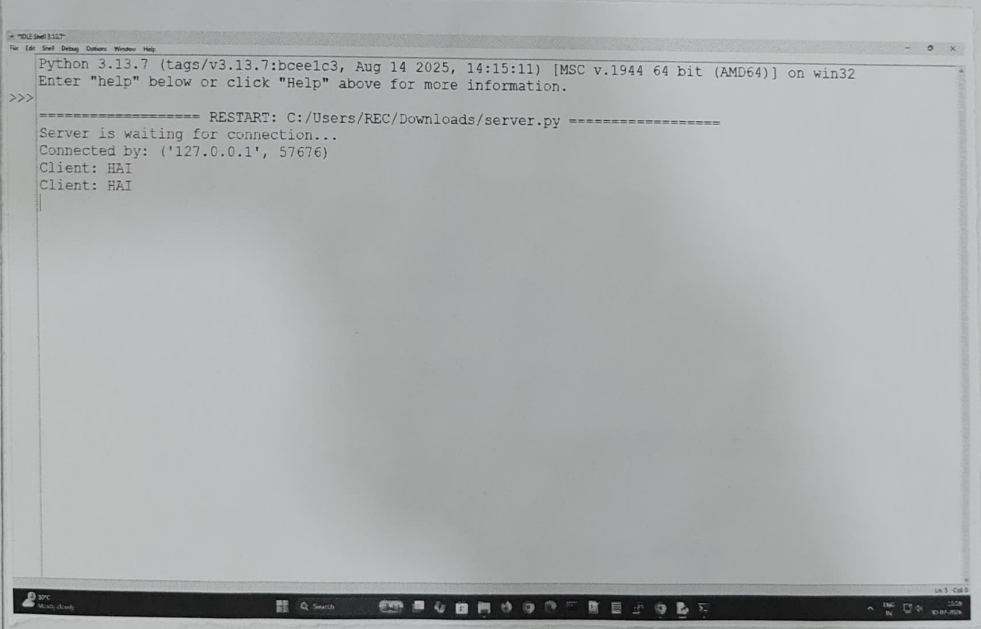
    client.send(message.encode())

    if message.lower() == "exit":
        break

    data = client.recv(1024).decode()
    print("Server:", data)

client.close()
```





RESULT:

Thus implementation of echo by using server and client is completed.

30/7/26

EXPERIMENT 6: BYTE STUFFING USING PYTHON

Aim:

To implement byte stuffing using Python

byte_stuffing.py - C:/Users/REC/AppData/Local/Programs/Python/Python310/byte_stuffing.p...

File Edit Format Run Options Window Help

```
# Byte Stuffing with Binary Representation

flag = input("Enter Flag Character: ")
escape = input("Enter Escape Character: ")
data = input("Enter Data: ")

stuffed = ""

for ch in data:
    if ch == flag or ch == escape:
        stuffed += escape
    stuffed += ch

frame = flag + stuffed + flag

print("\nOriginal Data      :", data)
print("Stuffed Data       :", stuffed)
print("Transmitted Frame  :", frame)

# Function to print binary values
def show_binary(text):
    for ch in text:
        print(ch, "->", format(ord(ch), '08b'))

print("\nBinary of Flag:")
show_binary(flag)

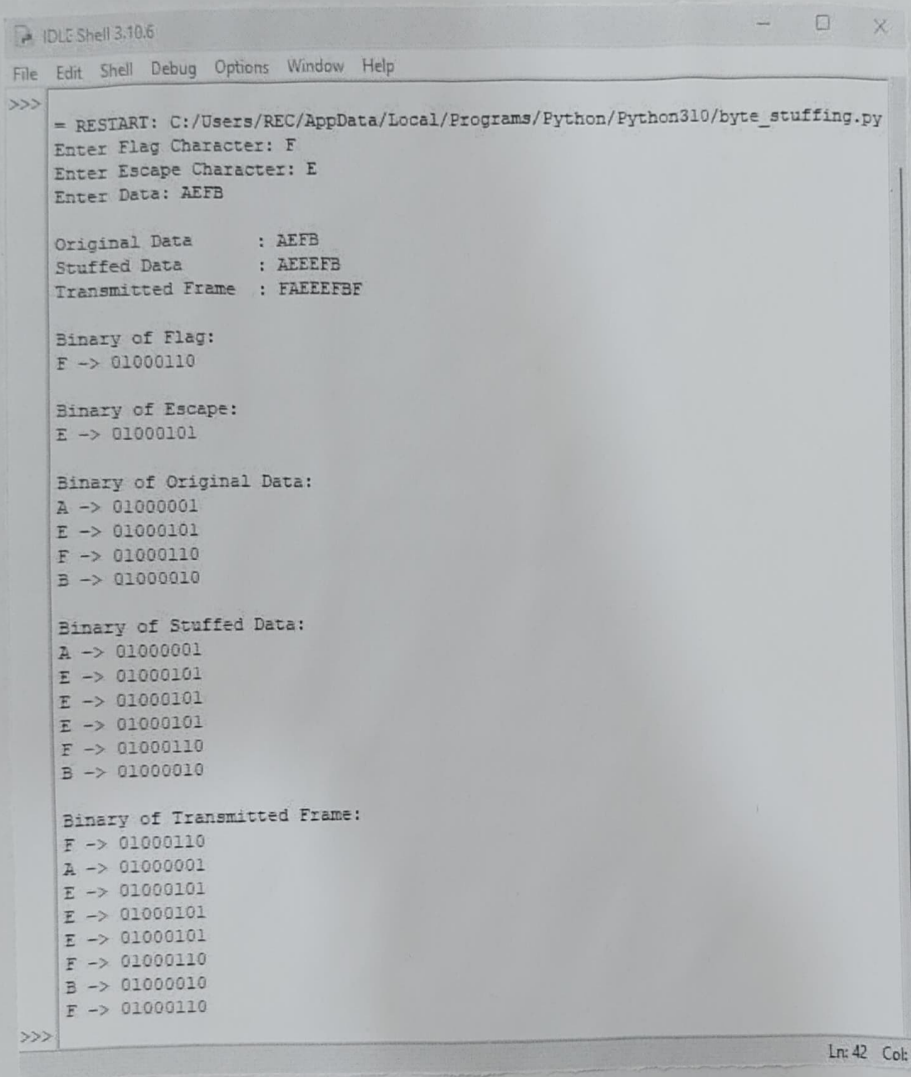
print("\nBinary of Escape:")
show_binary(escape)

print("\nBinary of Original Data:")
show_binary(data)

print("\nBinary of Stuffed Data:")
show_binary(stuffed)

print("\nBinary of Transmitted Frame:")
show_binary(frame)
```

Ln: 39 Col: 0

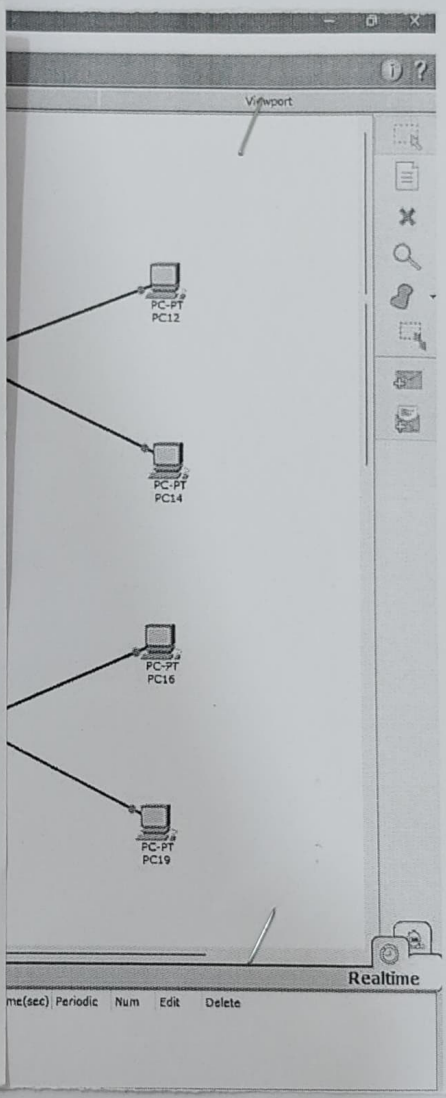


RESULT:

Thus, implementation of byte stuffing using python is executed successfully.

Exp no: 7
Date: 19/8/26

Implementation of subnetting in cisco packet Tracer simulator



Result:

Thus, the cisco packet tracer simulator is implemented successfully

server.py - C:/Users/REC/server.py (3.12.10)

File Edit Format Run Options Window Help

```
import socket
import threading

HOST = "0.0.0.0"
PORT = 5001


def receive_messages(conn):
    while True:
        try:
            message = conn.recv(1024).decode()

            if not message:
                break

            print(f"\nClient: {message}")
            print("You: ", end="", flush=True)

        except ConnectionResetError:
            break


server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(1)

print(f"Server listening on port {PORT}...")

conn, addr = server.accept()
print(f"Client connected: {addr}")

threading.Thread(
    target=receive_messages,
    args=(conn,),
    daemon=True
).start()

while True:
    message = input("You: ")

    if message.lower() == "exit":
        break

    conn.sendall(message.encode())

conn.close()
server.close()
```

client.py - C:/Users/REC/client.py (3.12.10)

File Edit Format Run Options Window Help

```
import socket
import threading

SERVER_IP = "127.0.0.1"
PORT = 5001


def receive_messages(sock):
    while True:
        try:
            message = sock.recv(1024).decode()

            if not message:
                break

            print(f"\nServer: {message}")
            print("You: ", end="", flush=True)

        except ConnectionResetError:
            break


client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((SERVER_IP, PORT))

print("Connected to server.")

threading.Thread(
    target=receive_messages,
    args=(client,),
    daemon=True
).start()

while True:
    message = input("You: ")

    if message.lower() == "exit":
        break

    client.sendall(message.encode())

client.close()
```

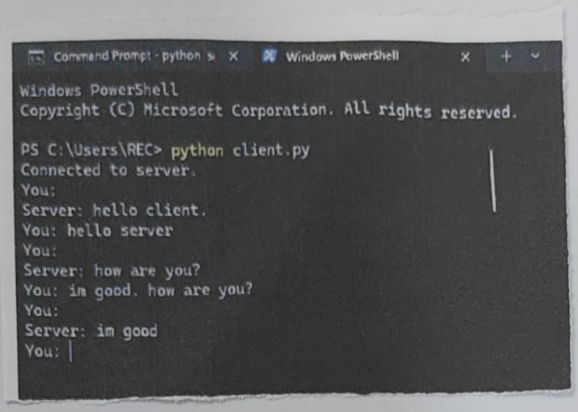
Result :-

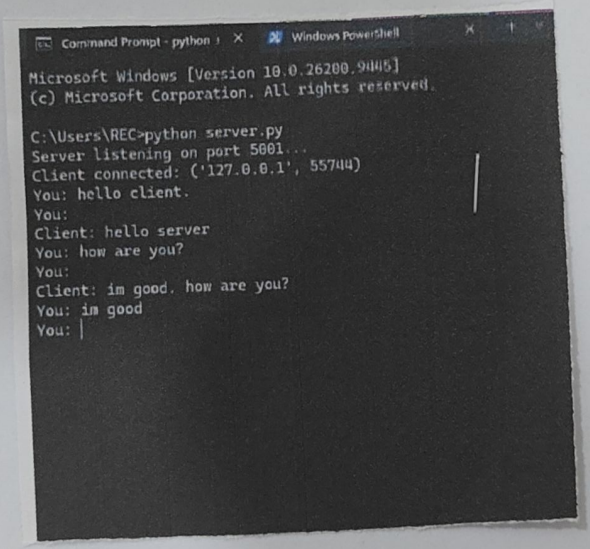
Thus, the client-server communication was successfully established & managed was exchanged successfully between client and server

Exp no:- 9 Distance vector Routing

Date: 16/9/26

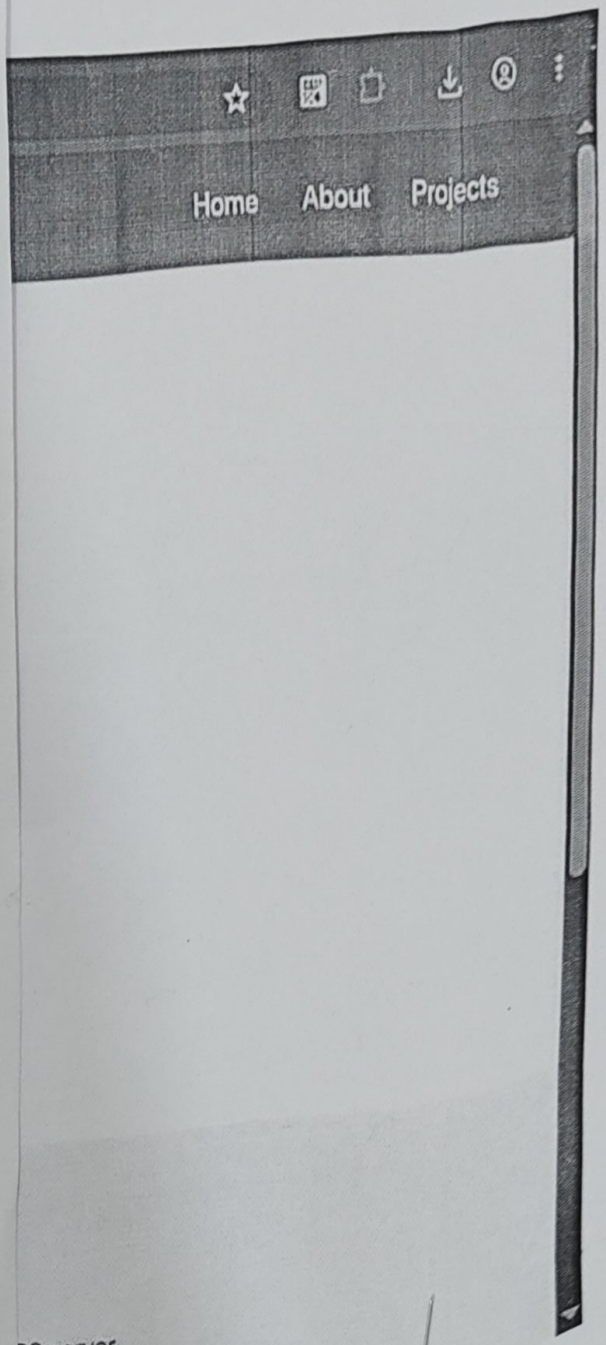
Aim: To implemat Distance vector routing





Result:

Thus to implement distance vector routing



C2 server.



Result:-

Thus, to deploy a web application of AWS server

Exp: 11 Experiment on Packet Capture Tool - wireshark

Aim :-

Capturing an analysis packets using wireshark tool.

1) AIM

Create a filter to display only TCP/UDP packets inspect the packets and provide the hour graph.

PROCEDURE

- Select LAN connection in wireshark
- Go to capture → option
- select Stop capture automatically after 100 packets
- Save the packets

2) AIM

Create a filter to display only ARP packets and inspect the packets

PROCEDURE

- Go to capture
- Set Stop Capture automatically after 100 packets
- Then click start capture
- Save the packets

3). AIM

Create a filter to display only DNS packets and provide the flow graph.

PROCEDURE

- Go to capture option
- Select stop capture automatically after 100 packets
- Then click start capture
- Save the packets

4).

AIM

Create a filter to display HTTP packets and repeat the packets.

PROCEDURE:

- Select LAN Connection in wireshark
- Go to Capture → option
- Then click start capture
- Save packets

Result :-

Thus, network packets were successfully captured and analysed using wireshark with different protocol filter.

Exp no :- 12 Discovery of Live host,
20/9/06 using ARP, ICMP, FCD, UDP

AIM :-

To discover live host in a network using Nmap scans such as ARP, ICMP, TCP, UDP on tryhackme platform.

OBJECTIVE :-

To understand how nmap identifies active systems before performing further network scanning.

SOFTWARE :-

Nmap
Tryhackme nMap lab
Computer with network success.

THEORY :-

nmap is free and open source network scanning tool used to discover live host & nmap user different live hosts using different protocols

1). ARP Scan - uses ARP Scans request to find hosts on subnet.

Observation :-

=) ARP request can discover hosts within same subnet.

=) ARP cannot cross router because it operates at layer

=) UDP ping can identify an active host.

Result :-

Thus, live host discovery using nmap was successfully studied and performed.

Expno :- 13
Date 27/9/24

Sliding Window Protocol

Aim :-

To write a program to implement flow control at data link layer using sliding window protocol and simulate the flow of frames from one node to another.

SENDER CODE :-

```
Message : Input ("Enter the message:")
window size = int(input ("Enter window size:"))
frames = []
for i, ch in enumerate(message):
        frames.append((i+1, ch))
print("\n frames created:")
for no, data in frames:
        print(f"frame {no}: {data}")
with open("sinda_buffer.txt", "w") as file:
    for no, data in frames
        file.write(f"{no} {data}\n")
print("\n frame stored in sender_buffer.txt")
test : 0
```

```
while base < len(frames)
    print("In current window:")
    for i in range(base, min(base + window_size,
                             len(frames))):
        print("[" + frames[i][0] + "=" + frames[i][1] + "]",
              end=" ")
```

RECEIVER CODE

```
expected_frames = 1
while True:
    try:
        with open("sender_buffer.txt", "r") as file:
            frames = file.readlines()
    except FileNotFoundError:
        print("Sender_Buffer.txt not found")
        break
    if not frames:
        print("No frames available")
        break
    print("\n Frame received from sender:")
    for line in frames:
        parts = line.split()
        if len(parts) != 2
```

Continue

```
frame_no = int(parts[1])
data = parts[2]
print(f"Frame {frame_no}: {data}")
if frame_no == expected_frame:
    print(f"Frame {frame_no} received")
with open("Receiver_Buffer.txt", "w") as out_file:

else:
    print("Wrong frame received")
    with open("Receiver_Buffer.txt", "w") as out_file:
    break
if expected_frames > len(frames):
    print("All frames received")
    break
input("Press Enter to check next frames")
```

Input

Enter the message: HELLO

Enter window size: 3

Output

Frames Created

Frame 1: H

Frame 2: E

Frames 3 : L

Frames 4 : L

Frames 5 : 0

Frames stored in sender. Buffer. bak

current window :

[1 : 1] [2 : E] [3 : L]

Frames Sent :

Run Receiver Program and press ENTER to continue

RECEIVER OUTPUT

Frames received from sender

Frame 1 : 1

Frame received correctly

ACK 1 Sent

Result :-

The Sliding window protocol was successfully implemented using Python.

Exp no: 14 Implementing Packet
8/10/06 Sniffing using Raw
Sockets

Aim:

To implement packet sniffing using RAW Sockets in python.

Algorithm:

1) Create a RAW Socket
2) Bind the Socket to network interface
3) Receive packets continuously
4) Read Packet headers
5) Display the protocol used
6) Continuous until user stops program

Input:

Network packets received through the network interface

Code:

```
import socket
S. socket. socket(socket.AF_INET, SOCKET, SOCK_RAW,
                  SOCKET_IPPROTO_ID)
S. bind(("0.0.0.0", 0))
S. setsocket(socket, IPPROTO.IP, socket.IP_HDRINCL, 1)
print {" Packets sniffing ended..."}
```

```
while True :
    Packet, addr = s.recvfrom (655565)
    ip_header : packet [0:20]
    iph : struct.unpack ("!BBHHHBBH4S4S", )

s_ip = socket.inet_ntoa (iph[8])
dest_ip = socket.inet_ntoa (iph[9])
print (" source IP :", src_ip)
print (" packet Length :", len (packet))
```

OUTPUT :

```
Packet sniffing started
Source IP : 192.168.1.10
Destination IP : 192.168.1.1
Packet Length : 77
```

Result :

Thus, a Raw socket program was successfully implemented the capture & display network packet.

Exp No:- 15 Analyse various types of
Date:- 8/10/26 server logs using
webalizer tool

AIM:

To analyse different types of server logs using the webalize tool.

ALGORITHM:

1) Install and open webalizer
2) Select a web server log file
3) Configure webalizer to read the log file
4) Run webalizer to process logs
5) Open generated HTML report

INPUT:

The web server log file such as access log

OUTPUT:

webalizer generates reports showing:

=> number of visitors
=> Number of requests
=> Most visited page
=> Data transferred
=> HTTP Status Code

Result:

The web logs were successfully analysed using webalizer and the web traffic statistics were generated.